Python Coding Assignment

10 original programs combining Python foundations and Day 2 topics. Complete all exercises independently and submit ONE Python source file.

Student: _____	Roll no.: _____
Class: AI Engineering / Week 1	Due date: _____

Submission instructions

- Write all 10 programs in a single file named `week1_python_assignment.py`.
- Clearly separate your answers using comments such as `# Program 01`, `# Program 02`, and so on.
- Use meaningful variable names and short comments to explain the important steps. Use functions/classes where a question requests them.
- Run and test the complete `.py` file before submission. Submit the .py file only (not a Colab link, screenshot, PDF, or ZIP).
- Use Python 3 and its standard library. Programs that create text, JSON, or database files may generate those files when executed.

PROGRAM 01 | Cinema Ticket Calculator

CONCEPTS: input(), int(), operators, if/else

- Ask for the customer's age and the number of tickets.
- Charge Rs. 300 per ticket for ages 12 and above; otherwise charge Rs. 180 per ticket.
- Display the ticket price and the total bill.

Check: Age 10, tickets 2 → total Rs. 360.

PROGRAM 02 | Name Tag Generator

CONCEPTS: strings, strip(), title(), upper(), len()

- Ask for first name and last name; remove extra spaces at the beginning and end.
- Display the full name in title case and the initials in uppercase (e.g. A.K.).
- Print the total number of letters, excluding the space between names.

Check: Input “ ali ” and “ khan ” → Ali Khan, A.K., 7 letters.

Coding tasks / Programs 03–06

PROGRAM 03 | Five-Day Step Tracker

CONCEPTS: for loop, range(), lists, sum(), max()

- Ask the user to enter the number of steps walked on each of 5 days. Save the values in a list.
- Show the total steps, highest daily steps, and average steps.
- Count how many days had 6,000 steps or more.

Check: For 2000, 6000, 8000, 5000, 9000 → qualifying days = 3.

PROGRAM 04 | Stationery Price Manager

CONCEPTS: dictionary, update, for loop, items()

- Create a dictionary containing prices of pen, notebook, and eraser. Choose reasonable starting prices.
- Ask which item to update and its new price. Change that item's price in the dictionary.
- Print each item and its updated price, then display the sum of the three prices.

Check: The updated item must appear with its new price.

PROGRAM 05 | Three-Subject Result Checker

CONCEPTS: functions, docstring, type hints, conditions, assert

- Write `average_marks(a: int, b: int, c: int) -> float` and add a one-line docstring.
- Ask for three subject marks, call the function, and print the average.
- Display Pass for average ≥ 50, otherwise Needs improvement. Add one `assert` test for the function.

Check: Marks 60, 70, 80 → average 70.0 and Pass.

PROGRAM 06 | Visitor Register

CONCEPTS: text files, with open(), w / a / r modes

- Ask for two visitors' names.
- Write the first name to `visitors.txt` using write mode; append the second name on a new line using append mode.
- Open the file in read mode and display both names.

Check: The text file must contain two lines.

Coding tasks / Programs 07–10

PROGRAM 07 | Movie Watchlist with JSON

CONCEPTS: list of dictionaries, json.dump(), json.load()

- Create a list of 3 movies. Each dictionary must have `title` (string) and `watched` (boolean).
- Save the list to `watchlist.json`, then reload it using `json.load()`.
- Print the title of the second movie and list the titles of all movies not yet watched.

Check: At least one movie should have watched = False.

PROGRAM 08 | Shared-Bill Calculator

CONCEPTS: float(), int(), try/except, ValueError

- Ask for the total restaurant bill and the number of friends sharing it.
- Calculate and display how much each person should pay, rounded to 2 decimal places.
- Use `try/except` to handle non-numeric input and display a friendly message when the number of friends is zero or negative.

Check: Bill 900, friends 3 → 300.00 each; friends 0 → friendly error.

PROGRAM 09 | Parcel Delivery Classes

CONCEPTS: OOP, __init__, attributes, methods, inheritance/overriding

- Create a `Parcel` class with item name and weight passed through `__init__`.
- Add a `delivery_fee()` method returning weight × 50.
- Create `ExpressParcel(Parcel)` and override `delivery_fee()` to return weight × 80. Create one object of each class and print both fees.

Check: For weight 2, normal fee = 100 and express fee = 160.

PROGRAM 10 | Book Club Database

CONCEPTS: sqlite3, CREATE TABLE, INSERT, SELECT, assert

- Use `sqlite3` to create or open `book_club.db` and create a `members` table with `id` and `name` columns.
- Insert two members using parameterized ? placeholders. Commit the changes.
- Read and print the member names using SELECT. Use an `assert` to check the query returns at least 2 rows. Close the database.

Check: Running the program again may add additional members; the test should still pass.

Marking & final checklist

Each program is worth 5 marks: functionality (3), correct results and basic validation (1), and clear code/comments (1). Total: 50 marks.

- One .py file contains clearly labelled answers to Programs 01–10.
- All required functions, classes, file operations, and database operations are present.
- The file runs with Python 3 and prompts the user clearly where input is required.
- No pre-written solutions are included in this assignment brief.